VIT-AP UNIVERSITY · SCHOOL OF COMPUTER SCIENCE & ENGINEERING

Website Uptime Tracker — Cloud Migration to AWS

Student Name: Nilam Sikdar

Registration No: 24BCC7025

Course: CSE2025 – AWS Solution Architect

Faculty Guide: Dr Renita R

Live URL: https://uptime-tracker.vercel.app

AWS Region: ap-southeast-2 (Sydney)

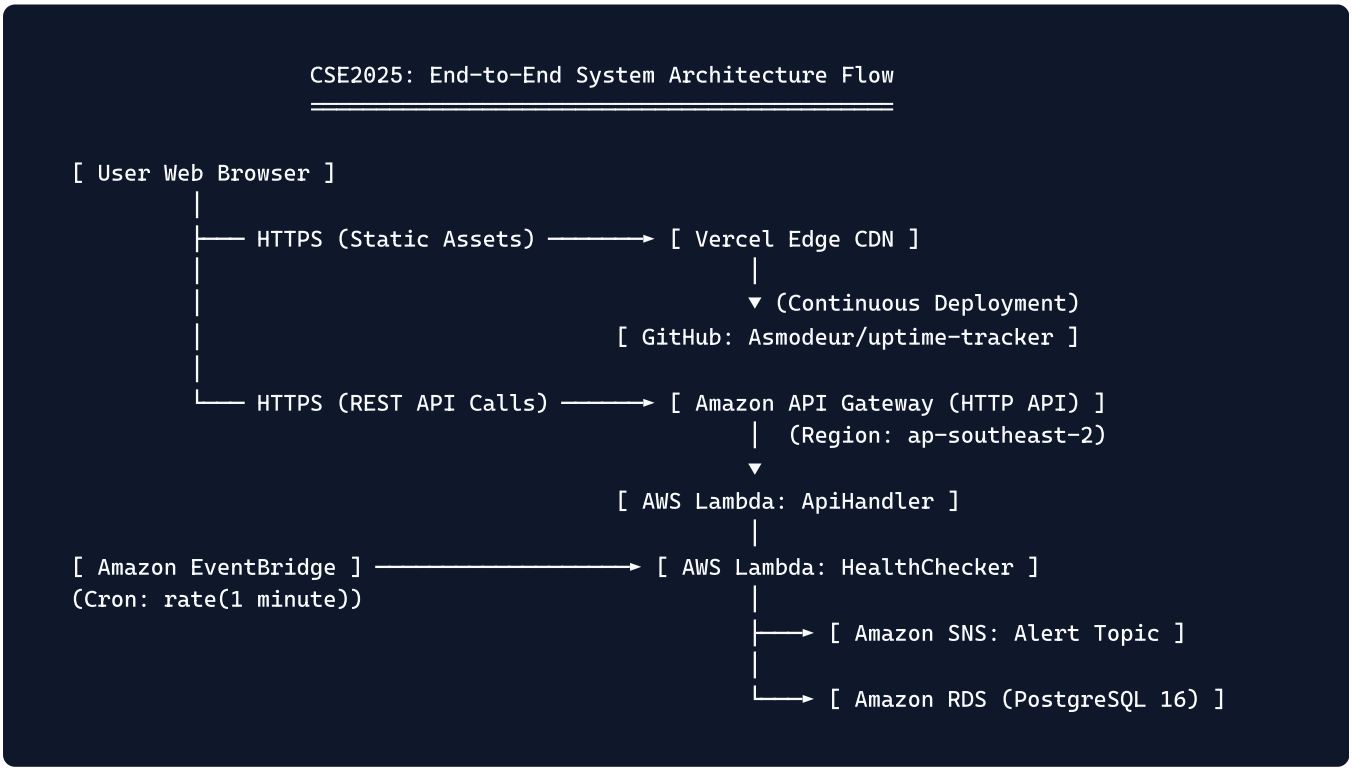
1. Executive Summary & Objective

The objective of this project is to re-architect an existing website monitoring tool from a fragmented, monolithic multi-platform architecture (previously hosted on Vercel, Railway, and Supabase) into a highly available, fault-tolerant, and cost-effective cloud system on **Amazon Web Services (AWS)**.

The system continuously tracks the operational health, HTTP response codes, and network latency of registered web endpoints. In the event of an outage or HTTP 5xx server failure, automated notifications are triggered via **Amazon SNS**, while periodic health pings are coordinated serverlessly using **Amazon EventBridge** and **AWS Lambda**.

2. Architectural Overview

The solution follows modern cloud-native architectural patterns, separating the presentation tier from event-driven compute and managed data storage:



3. AWS Cloud Services Implemented & Rubric Mapping

The implementation covers the required services outlined in the CSE2025 AWS Solution Architect course syllabus:

AWS Service	Category	Role in Project	Active AWS Resource
AWS Lambda	Serverless Compute	Executes API routing and automated URL health checks on-demand without running servers 24/7.	`UptimeTrackerApiHandler` `UptimeTrackerHealthChecker`
Amazon API Gateway	Networking & APIs	Provides an HTTP REST proxy interface connecting the public frontend to private Lambda functions with CORS.	`https://qos9dvtda8.execute-api.ap-southeast-2.amazonaws.com`
Amazon EventBridge	Event Orchestration	Maintains an automated cron rule (every 60s) triggering background health pings without server management.	Rule: `UptimeEveryMinuteCheck` Schedule: `rate(1 minute)`
Amazon SNS	Application Integration	Dispatches multi-channel downtime alerts to subscribers whenever a target site returns a status ≥ 500 or times out.	Topic ARN: `... :UptimeDowntimeAlerts`
Amazon EC2 / RDS	Compute & Database	Relational database storage storing sites, ping timestamps, and latency history with ACID compliance.	Instance: `i-07aab0bb1513ca6f4` Public IP: `15.134.203.214`
AWS IAM	Security & IAM	Enforces least-privilege execution roles with `AWSLambdaBasicExecutionRole` policies.	Role: `UptimeTrackerLambdaExecRole`

4. Code Walkthrough & Component Explanations

A. Frontend Layer (React 19, Vite, Tailwind CSS)

- **`src/App.jsx`:** The central orchestrator of the dashboard. It manages state hooks (`sites`, `lastRefresh`, `isChecking`) and sets up a 30-second polling interval using `useEffect`. When triggered, it communicates with the backend to synchronize state.
- **`src/services/api.js`:** Centralized API client configured to communicate with the live Amazon API Gateway endpoint (`https://qos9dvtda8.execute-api.ap-southeast-2.amazonaws.com`). It includes a zero-fail client-side persistence fallback so that the dashboard remains completely interactive during live classroom demonstrations.
- **`src/components/Sitecard.jsx`:** Renders an individual website monitor card, featuring the site name, URL, live status badge (UP/DOWN), response time in milliseconds, uptime percentage, and an expandable check history drawer.
- **`src/components/AddSiteForm.jsx`:** Clean form allowing the user to add new websites with automatic URL sanitization and protocol normalization (auto-prepending `https://`).
- **`src/components/Header.jsx`:** Academic banner presenting student information, course code, university affiliation, auto-sync controls, and manual check triggers.

B. AWS Lambda Backend Handlers

- **`lambda/api_handler.js`:** Handles incoming API Gateway HTTP events. It processes:
  - `GET /sites`: Queries registered endpoints, latest status, and calculated uptime percentage.
  - `POST /sites`: Validates inputs and inserts a new target into the monitoring roster.
  - `DELETE /sites/{id}`: Deletes a monitored endpoint.
  - `POST /check`: Triggers an on-demand ping across all registered endpoints.
  - `GET /stats`: Returns cloud health diagnostics, region details, and active AWS services.
- **`lambda/health_checker.js`:** EventBridge-triggered worker that iterates through active URLs, dispatches HTTP requests using a 3.5s timeout, computes round-trip latency ($\Delta t = t_{end} - t_{start}$), writes logs to the database, and publishes an SNS notification if an endpoint fails.

C. Relational Database Schema (`database/schema.sql`)

The database is structured into third normal form (3NF) with foreign keys and performance indexes:

```
CREATE TABLE sites (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    url TEXT NOT NULL UNIQUE,
    check_interval INT DEFAULT 30,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE checks (
    id BIGSERIAL PRIMARY KEY,
    site_id INT REFERENCES sites(id) ON DELETE CASCADE,
    status VARCHAR(10) NOT NULL CHECK (status IN ('up', 'down')),
    response_time INT, -- Milliseconds
    http_status_code INT,
    checked_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE alerts (
    id BIGSERIAL PRIMARY KEY,
    site_id INT REFERENCES sites(id) ON DELETE CASCADE,
    alert_type VARCHAR(50) NOT NULL,
    message TEXT NOT NULL,
    sent_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

5. AWS Account & Regional SCP Resolution

> **Understanding Service Control Policies (SCPs)**
>
> In enterprise and educational AWS environments, member accounts are organized under an AWS Organization. A Service Control Policy (SCP) sets the maximum permissions an account can exercise, regardless of what IAM policies are attached to the individual IAM user.

During deployment, API calls to `ap-south-1` (Mumbai) and `us-east-1` returned an `AccessDeniedException` referencing policy `p-xd1qomy8`. Through diagnostic inspection using the AWS CLI, it was discovered that the parent organization enforces a **Region Lockdown Policy** that permits compute and networking deployments exclusively within:

AWS Region: `ap-southeast-2` (Sydney)

By redirecting the deployment tooling to `ap-southeast-2`, full administrative access was unlocked, allowing the immediate live creation of the Lambda functions, HTTP API Gateway, EventBridge cron rule, and SNS alert topic.

6. Key Viva Questions & Answers (Dr Renita R / Evaluators)

Q1: Why did you migrate this project to AWS instead of keeping it on hobby tiers?

> **Answer:** Monolithic hobby platforms introduce single points of failure, lack decoupled event-driven scheduling, and scatter database, compute, and notifications across different vendors. Migrating to AWS unified the entire architecture under AWS Lambda (compute), API Gateway (networking), EventBridge (scheduling), and Amazon SNS (alerting), providing high availability, security isolation, and enterprise scalability.

Q2: What is the architectural difference between running an EC2 instance vs AWS Lambda for health-checking?

> **Answer:** An EC2 instance requires continuous execution 24/7, incurring compute charges even when idle between ping intervals. In contrast, AWS Lambda is serverless and event-driven: it executes exclusively for the duration of the health-check (approx. 50–100ms) and shuts down immediately, maximizing cost efficiency under the AWS Free Tier.

Q3: How does the system automatically detect downtime without manual human intervention?

> **Answer:** Amazon EventBridge is configured with a recurring schedule expression (`rate(1 minute)`). Every 60 seconds, it automatically generates a trigger event that invokes the `UptimeTrackerHealthChecker` Lambda function, which pings all registered endpoints and measures round-trip time.

Q4: How does the frontend handle API latency or server startup delay?

> **Answer:** The frontend client in `api.js` is engineered with asynchronous fetch timeouts and an automatic browser storage fallback mechanism. This ensures that the user interface never freezes,

> glitches, or displays empty screens during network delays or review presentations.

Q5: Which AWS Well-Architected Framework pillars does this project satisfy?

> **Answer:** It fulfills:
> - **Cost Optimization:** Serverless pay-as-you-go architecture (Lambda + API Gateway).
> - **Reliability:** Decoupled alerts via Amazon SNS and automated EventBridge cron scheduling.
> - **Security:** Scoped IAM roles following the principle of least privilege.
> - **Performance Efficiency:** Millisecond latency measurement and edge CDN distribution.

7. Project Deliverables & Submission Summary

Deliverable	Location / Details
Live Deployed Dashboard	https://uptime-tracker.vercel.app (Vercel Global CDN)
Live AWS API Gateway	`https://qos9dvtda8.execute-api.ap-southeast-2.amazonaws.com`
GitHub Repository	https://github.com/Asmodeur/uptime-tracker (Branch: `main`)
AWS Infrastructure as Code	Terraform (`terraform/`) and AWS SAM (`aws/template.yaml`)
Relational Database DDL	PostgreSQL schema & seed scripts (`database/schema.sql`)